

Práctica 1 de Lógica Computacional

Introducción a Prolog

Erik Figueiral Alonso

erik.figueiral.alonso@uvigo.gal

21 de julio de 2026

Horario · Erik Figueiral Alonso

Lunes **09:00–10:30**

Jueves **15:00–16:30**

Laboratorio de Ciencia Computacional

3 horas concertadas · erik.figueiral.alonso@uvigo.gal

Qué vamos a aprender y practicar

Cargar un programa, interpretar sus respuestas y escribir relaciones recursivas sencillas.

Bloque	Contenido
1 · Preparación y lenguaje	Paradigma, entorno, sintaxis y semántica
2 · Consultas y resolución	Preguntas, unificación y árboles
3 · Ejercicios	Naturales, suma, paridad, igualdad y aritmética

Describimos las propiedades; el motor resuelve

Nosotros: PROPIEDADES del problema.
El MOTOR de inferencia busca las soluciones.

Incógnitas lógicas, sin reasignaciones ni entradas/salidas fijas.

«Quien tiene un amigo tiene un tesoro».

Lo que describimos	Ejemplo
Hecho	Ana tiene un amigo.
Regla	Si alguien tiene un amigo, tiene un tesoro.
Pregunta → deducción del motor	¿Quién tiene un tesoro? → Ana.

PSR (problemas de satisfacción de restricciones), IA y planificación:
definimos qué debe cumplirse; el motor busca cómo satisfacerlo.

Equivalencia: qué se demostró y quién

Imperativo · Funcional · Lógico

Resultado	Autor
Cálculo lambda y máquinas de Turing tienen la misma capacidad de cálculo.	Alan Turing · 1937
Las cláusulas de Horn permiten expresar cualquier función computable.	Sten-Åke Tärnlund · 1977

La equivalencia es de capacidad, no de eficiencia.

Antes de empezar: abrir SWI-Prolog

En los equipos Ubuntu del aula debería estar instalado.

```
swipl
```

En la terminal → aparece ?-

Si no está instalado en vuestro Ubuntu:

Terminal del sistema

```
sudo apt update  
sudo apt install swi-prolog
```

Último recurso: swish.swi-prolog.org

Versión web: no la recomiendo para seguir esta práctica.

Cargar, preguntar y salir

Guardamos natural.pl en la carpeta desde la que abrimos Prolog.

Consola de Prolog

```
?- consult(natural).  
true.
```

1 Cargar el archivo

Consola de Prolog

```
?- hombre(adan).  
true.
```

2 Hacer una pregunta

3 Salir: halt. Si escribimos halt sin punto, Prolog espera.

El programa de ejemplo aparece a continuación. Tras editar: guardar y volver a cargar con consult(natural).

Un nombre no es una variable

¿Qué cambia al escribir una mayúscula?

natural

'natural'

El mismo nombre
(un átomo)

Natural

'Natural'

Natural: variable
'Natural': nombre

Para cargar el fichero: `consult('natural.pl').`

Mayúscula o _ al inicio: variable. Comillas simples: nombre literal.

Primero, una tabla de sintaxis y semántica

La construiremos para cada relación que programemos.

Sintaxis	Semántica que acordamos	Firma
hombre(P)	P es un ser humano	hombre/1
mortal(P)	P es mortal	mortal/1

El nombre no aporta el significado: lo fijamos nosotros.

La firma indica el nombre y el número de argumentos.

La sintaxis de un programa lógico

Un programa lógico puro es un conjunto de cláusulas de Horn definidas.

Forma	Cómo se lee
$p.$	Hecho o axioma: p se acepta sin premisas.
$p \text{ :- } q, r.$	Regla: p es cierto si q y r son ciertos.
$?- p.$	Pregunta al intérprete: ¿puede demostrar p ?

Una conclusión (cabeza), cero o más premisas (cuerpo).

$p \vee q \leftarrow r$ no es Horn: contiene dos conclusiones positivas.

La consulta no se escribe como un hecho del programa. Aquí usamos el fragmento lógico puro.

Nuestro primer programa

Archivo natural.pl

```
hombre(adan).           % c1: hecho
hombre(eva).            % c2: hecho

mortal(P) :- hombre(P). % c3: regla
```

Para toda P: si P es un ser humano, P es mortal.

P es una variable. adan y eva son átomos.

% comenta una línea. /* ... */ delimita un comentario.

Preguntar y pedir otra respuesta

Consola de Prolog

```
?- hombre(adan).  
true.
```

```
?- hombre(P).  
P = adan
```

•

Me quedo con esta

;

Busco otra: P = eva

Primero: punto para terminar la consulta. Después: . o ;

Unificar: hacer coincidir dos términos

hombre(P) hombre(adan)

$\theta = \{P = \text{adan}\}$

La sustitución hace idénticos los dos términos.

hombre(P) mortal(adan)

No unifican: sus nombres son diferentes.

Resolución: demostrar mortal(adan)

{mortal(adan)}

c3, $\theta_1 = \{P1 = \text{adan}\}$

{hombre(adan)}

c1, $\theta_2 = \{ \}$

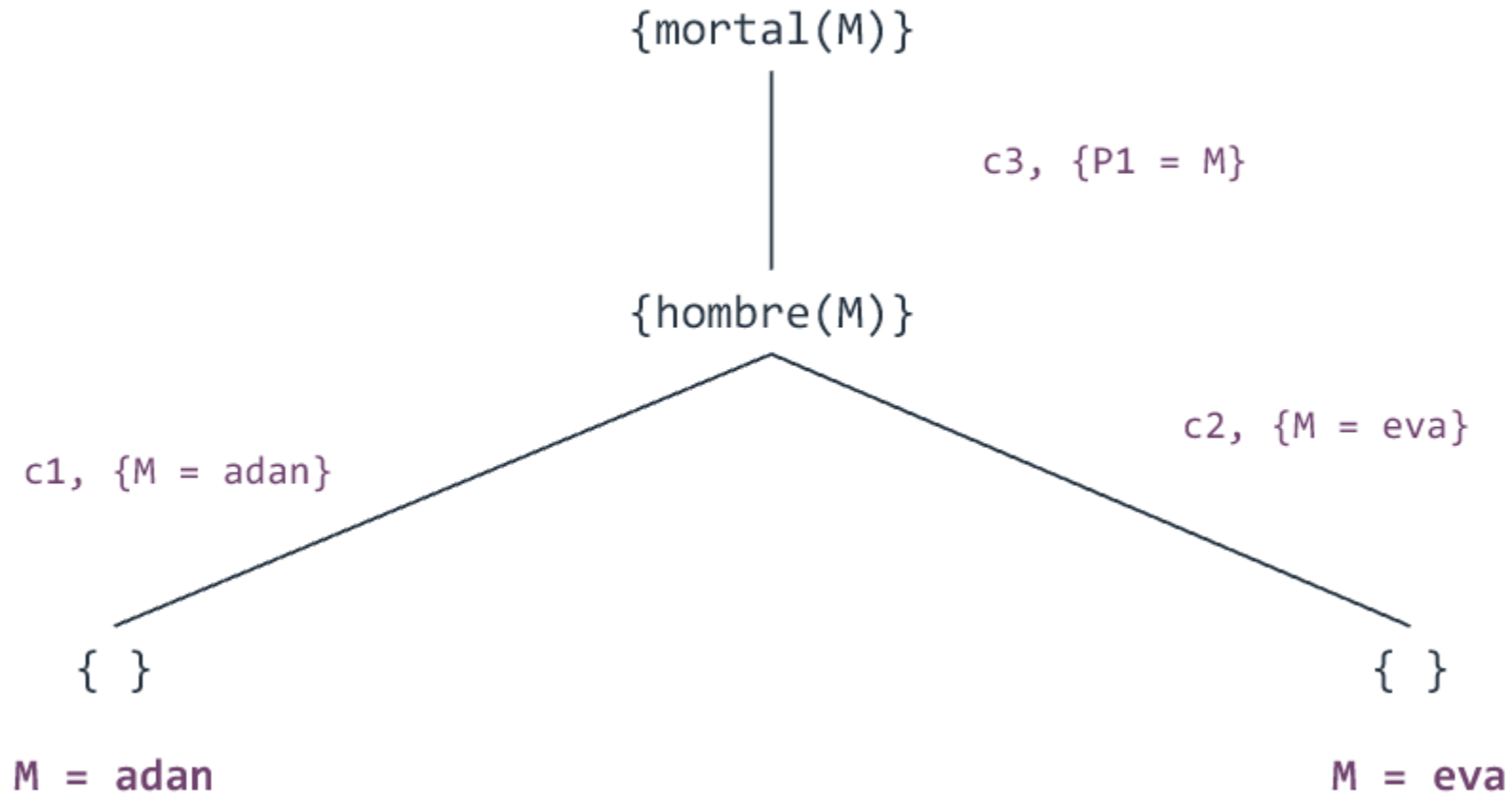
{ }

Éxito

Cada nodo: objetivos pendientes.
Cada rama: cláusula y unificador.

Resolventes y renombramiento de variables: apuntes, §3.1, pp. 8–11.

Resolución: todas las respuestas de mortal(M)



`{ }` es la resolvente vacía. No quedan objetivos. Cada hoja de éxito da una respuesta.

Naturales: sintaxis, semántica y representación

Sintaxis	Semántica	Firma
0	El número cero	constante
s(X)	El siguiente de X	s/1, funtor
nat(N)	N es un natural	nat/1, predicado

0
 $s(0)$
 $s(s(0))$
 $s(s(s(0)))$

s(X) construye un término. nat(N) expresa una propiedad.

Los naturales: un axioma y una regla

Archivo natural.pl

```
nat(0).                                % c1  
nat(s(X)) :- nat(X).                  % c2
```

El cero es natural.

Si X es natural, su siguiente también lo es.

Axioma → caso base. Regla recursiva → problema más pequeño.

¿Qué consultas tendrán éxito?

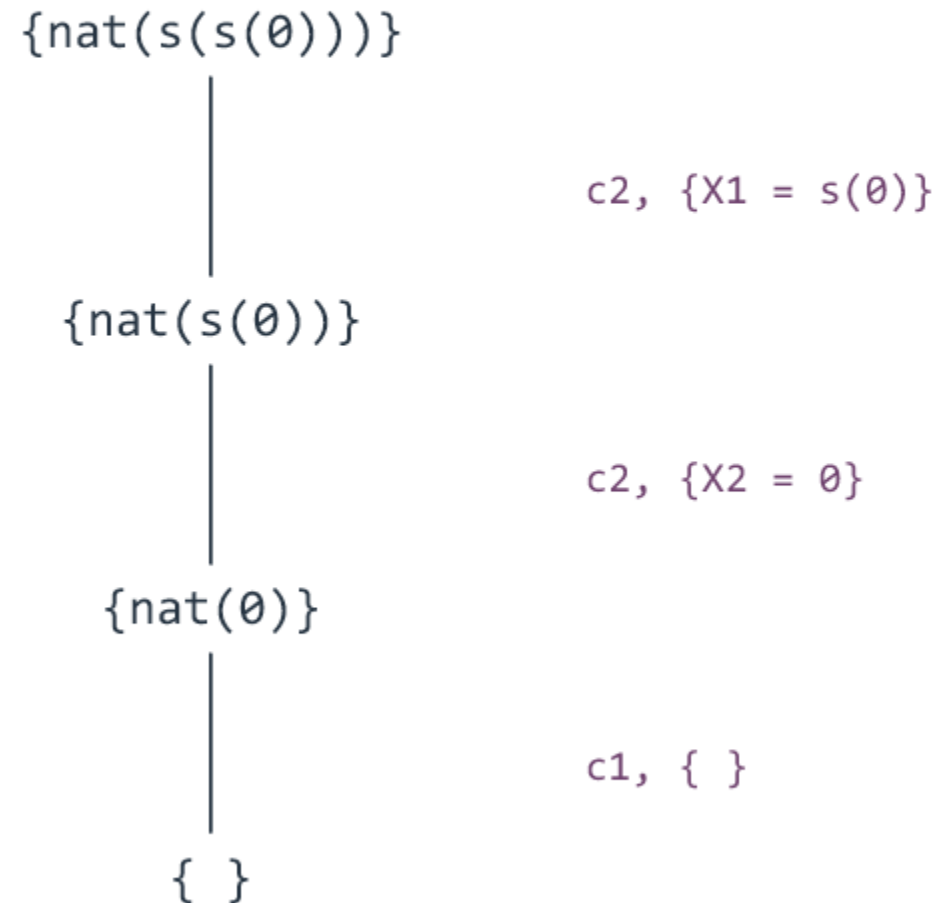
Responded antes de ejecutar.

Consola de Prolog

```
?- nat(0).  
?- nat(s(0)).  
?- nat(1).
```

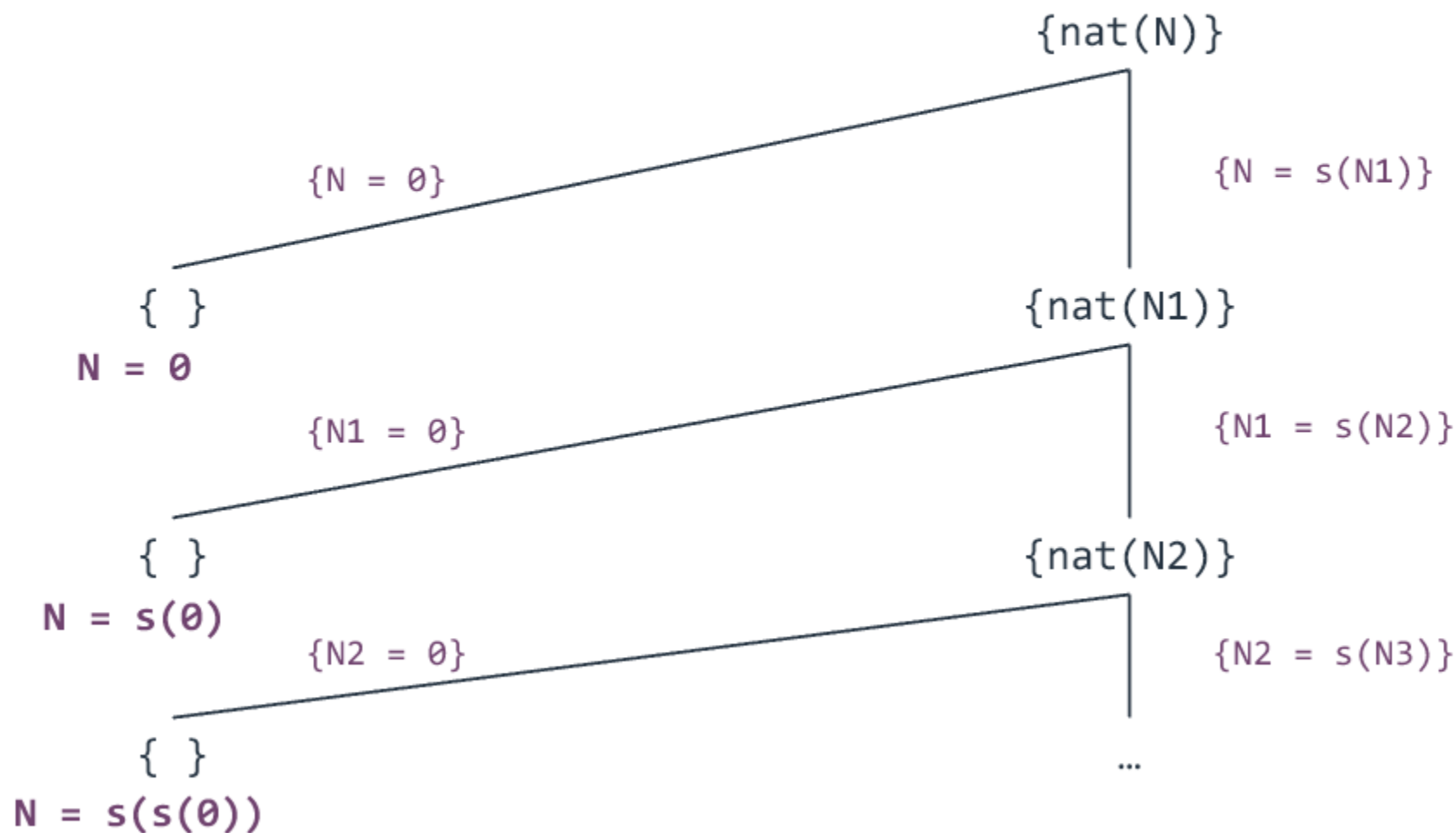
¿Y nat(S(0))? ¿Qué cambia con esa mayúscula?

Resolución: comprobar el natural dos



`nat(1)` falla: el término 1 no es 0 ni tiene la forma `s(X)`. `nat(S(0))` es un error de sintaxis.

Resolución: generar naturales



En cada paso renombramos las variables de la regla. La rama derecha continúa indefinidamente.

En el intérprete: una respuesta y otra

```
?- nat(N).
```

SWI-Prolog 10.0.2 · Salida de ejecución

```
0  
s(0)  
s(s(0))  
s(s(s(0)))  
s(s(s(s(0))))
```



Otra



Parar

Cada respuesta llega. La enumeración completa no termina.

Ejercicio: definir la suma

Sintaxis	Semántica	Firma
<code>suma(X,Y,Z)</code>	$X + Y = Z$	<code>suma/3</code>

Consola de Prolog

```
?- suma(0,s(0),Z).
?- suma(s(0),s(0),Z).
```

Escribid un axioma y una regla. ¿Qué debería responder?

Razonad primero: todavía no mostramos ni la definición ni las respuestas.

Suma: caso base y paso recursivo

Archivo natural.pl

```
suma(0,Y,Y). % c1
suma(s(X),Y,s(Z)) :- suma(X,Y,Z). % c2
```

$$0 + Y = Y$$

$$s(X) + Y = s(Z) \quad \text{si} \quad X + Y = Z$$

¿Y si el axioma fuera $\text{suma}(X,0,X)$.

sin cambiar la regla? Probad $1 + 1$. ¿Dónde se detiene la prueba?

¿Qué podemos preguntar a la misma relación?

Consola de Prolog

```
?- suma(s(0),s(0),Z).
```

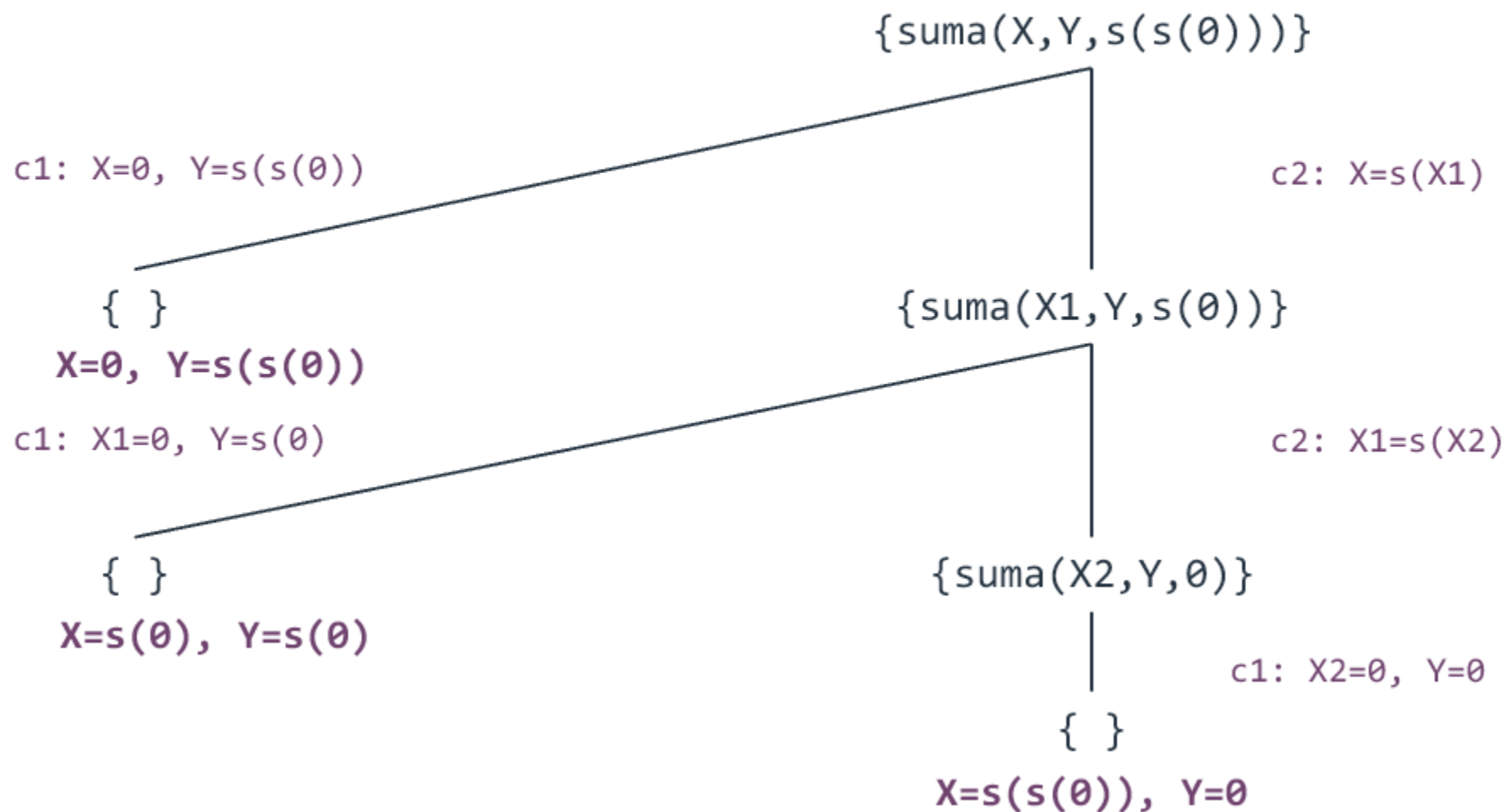
```
?- suma(s(0),Y,s(s(0))).
```

```
?- suma(X,s(0),s(s(0))).
```

Predecid cada respuesta y explicad cómo se obtiene.

¿Qué conocemos y qué queremos averiguar en cada consulta?

Resolución: parejas que suman dos



Ramas rotuladas con las sustituciones relevantes. Cada aplicación utiliza variables frescas.

Una respuesta puede describir infinitos casos

Consola de Prolog

```
?- suma(0,X,Y).  
X = Y.
```

$$0 + 0 = 0$$

$$0 + s(0) = s(0)$$

$$0 + s(s(0)) = s(s(0))$$

¿Por qué no hace falta enumerar todos esos pares?

Ejercicio: números pares

Sintaxis	Semántica	Firma
par(N)	N es un natural par	par/1

Consola de Prolog

```
?- par(0).
?- par(s(0)).
?- par(s(s(0))).
```

Escribid el caso base y una regla recursiva.

Después: consult(natural). y par(N). Pedid varias respuestas con ;

Solución: números pares

Archivo natural.pl

```
par(0).  
par(s(s(X))) :- par(X).
```

Consulta en Prolog	Respuesta
?- par(0).	true
?- par(s(0)).	false
?- par(s(s(0))).	true

Ejercicio: números impares

Sintaxis	Semántica	Firma
<code>impar(N)</code>	N es un natural impar	<code>impar/1</code>

Consola de Prolog

```
?- impar(0).  
?- impar(s(0)).  
?- impar(s(s(s(0))))).
```

¿Qué cambia respecto a `par/1`? ¿Qué se mantiene?

Escribid la definición, guardad el fichero, cargadlo y probad `impar(N)`.

Solución: números impares

Archivo natural.pl

```
impar(s(0)).  
impar(s(s(X))) :- impar(X).
```

Consulta en Prolog	Respuesta
?- impar(0).	false
?- impar(s(0)).	true
?- impar(s(s(s(0)))).	true

Ejercicio: pueden ser iguales

Sintaxis	Semántica	Firma
<code>igual(X,Y)</code>	X e Y pueden unificarse	<code>igual/2</code>

Consola de Prolog

```
?- igual(f(X),f(5)).  
?- igual(f(5),s(5)).  
?- igual(5,X).
```

¿Se puede definir con un solo axioma?

Solución: igualdad por unificación

Archivo natural.pl

```
igual(X,X).
```

Pregunta	Respuesta
<code>igual(f(X),f(5)).</code>	<code>X = 5</code>
<code>igual(f(5),s(5)).</code>	<code>false</code>
<code>igual(5,X).</code>	<code>X = 5</code>

Los dos argumentos deben poder convertirse en el mismo término.

Ejercicio: multiplicación

Sintaxis	Semántica	Firma
<code>mult(X,Y,Z)</code>	$X \times Y = Z$	<code>mult/3</code>

Archivo natural.pl

```
mult(0,Y,____).
mult(s(X),Y,W) :- mult(X,Y,Z), ____.
```

¿Qué hay que sumar al producto $X \times Y$?

Solución: multiplicación

Archivo natural.pl

```
mult(0,_,0).
mult(s(X),Y,W) :-
    mult(X,Y,Z), suma(Z,Y,W).
```

$$0 \times Y = 0$$

$$s(X) \times Y = X \times Y + Y$$

¿Hace falta añadir el neutro `mult(1,X,X)`? ¿Qué pasaría?

En el programa de Peano se escribe `mult(s(0),X,X).` `_` es la variable anónima del axioma.

4 × 2: bajar hasta el axioma

Pregunta: `mult(4,2,R4)`.

Objetivo seleccionado	Se sustituye por estos objetivos
<code>mult(4,2,R4)</code>	<code>mult(3,2,R3), suma(R3,2,R4)</code>
<code>mult(3,2,R3)</code>	<code>mult(2,2,R2), suma(R2,2,R3)</code>
<code>mult(2,2,R2)</code>	<code>mult(1,2,R1), suma(R1,2,R2)</code>
<code>mult(1,2,R1)</code>	<code>mult(0,2,R0), suma(R0,2,R1)</code>
<code>mult(0,2,R0)</code>	Axioma: $R0 = 0$

Resolvemos primero cada mult. Las sumas quedan pendientes.

Usamos numerales para leer la traza: 4 abrevia $s(s(s(s(0))))$. No es código para copiar con estos literales.

4 × 2: resolver las sumas pendientes

Hemos llegado al axioma: $0 \times 2 = 0$.

Producto que recuperamos	Suma pendiente	Resultado
1×2	suma(0,2,R1)	R1 = 2
2×2	suma(2,2,R2)	R2 = 4
3×2	suma(4,2,R3)	R3 = 6
4×2	suma(6,2,R4)	R4 = 8

Seguimiento: trace. lo activa; notrace. lo desactiva.

$4 \times 2 = 8$ porque $3 \times 2 = 6$ y $6 + 2 = 8$.

$1 \times X$ ya se deduce: añadir mult(s(0),X,X). repite pruebas para naturales. Los numerales abrevian Peano.

Ejercicio: potencia

Sintaxis	Semántica	Firma
$\text{exp}(X,Y,Z)$	X elevado a Y es Z	exp/3

Archivo natural.pl

```
exp(X,0,____).
exp(X,s(Y),W) :- exp(X,Y,Z), ____.
```

¿Cuál es el caso base? ¿Cómo reutilizamos mult/3?

Solución: potencia

Archivo natural.pl

```
exp(_, 0, s(0)).
exp(X, s(Y), W) :-
    exp(X, Y, Z), mult(X, Z, W).
```

```
?- exp(s(s(0)), s(s(0)), Z).
Z = s(s(s(s(0)))).
```

exp(X,s(0),X). sería redundante: ya se deduce.

Convención de esta definición sobre naturales: también asigna 1 a 0 elevado a 0.

Ejercicio final: el doble de un natural

Sintaxis	Semántica	Firma
<code>doble(X,Y)</code>	$Y = X + X$	<code>doble/2</code>

Archivo `natural.pl`

```
doble(X,Y) :- _____.
```

Consola de Prolog

```
?- doble(s(0),Y).
?- doble(X,s(s(0))).
```

Completad la regla y predecid las dos respuestas.

Usad únicamente `suma/3`. Firma: `doble/2`. Reutilizamos `natural.pl`.

Solución: una relación, dos direcciones

Archivo natural.pl

```
doble(X,Y) :- suma(X,X,Y).
```

Consola de Prolog

```
?- doble(s(0),Y).
```

```
Y = s(s(0)).
```

```
?- doble(X,s(s(0))).
```

```
X = s(0).
```

En ambos casos resolvemos $X + X = Y$. Solo cambia qué valor queremos averiguar.